Chapter 13: Advanced Techniques and Performance

Download this chapter as a PDF for offline reading:
Chapter-13-Advanced-Techniques-and-Performance.pdf

Queries that are correct can still be too slow, and flows that should exist sometimes stop for reasons that are not obvious. This chapter covers two practical skills: writing predicates that the evaluator can run efficiently, and debugging the places where data flow or taint tracking stops.

Efficient Predicates

The CodeQL evaluator is fast, but it still rewards clear, direct predicates. A few habits make a large difference on real databases.

Prefer direct relationships

When a library already provides a predicate that answers your question, use it. Walking the AST by hand with nested `exists` is usually slower and harder to read.

```
// Prefer this
predicate isEmptyFunction(Function f) {
  not exists(f.getAStmt())
}

// Over repeatedly inlining the same condition
```

(See `examples/chapter13/efficient-predicate.ql`.)

Extract common conditions

If the same filter appears in several places, extract it into a predicate (as shown in Chapter 5). The evaluator can often share the work, and the query becomes easier to maintain.

Avoid unnecessary Cartesian products

A `from` clause that binds many independent variables can explode. Constrain the variables as early as possible with `where` conditions that relate them.

Use the right flow primitive

Local flow (`DataFlow::localFlow` or `TaintTracking::localTaint`) is cheaper than a full configuration. If the behavior you care about is confined to one function, stay local.

Measure

On a large database, time the query:

```
codeql query run --database=my-db my-query.ql
```

The CodeQL extension for VS Code and the CLI both report timing. When a query is slow, the usual culprits are an overly broad `exists`, a missing early filter, or an accidental use of global flow when local flow would suffice.

Partial-Flow Debugging

A path query that returns no results is not always wrong. The flow may stop at a library call that is not modeled, at a barrier you added, or at a point where the value is transformed in a way the library does not yet understand.

Locate the break

1. Confirm the source and sink exist independently.
2. Replace the full path query with a local-flow check between the source and an intermediate node you expect to reach.
3. Move the intermediate node forward until the local flow fails. The failure point is where the model is missing or a barrier is active.

Skeleton (see `examples/chapter13/partial-flow-debug.ql`):

```
from DataFlow::Node source, DataFlow::Node mid
where
  source instanceof RemoteFlowSource and
  DataFlow::localFlow(source, mid) and
  mid = any(Expr e).flow()
select mid, "Reached this intermediate node from a remote source."
```

Common reasons flow stops

- A library function is not modeled (add a summary or an `isAdditionalFlowStep`).
- A barrier or sanitizer is matching more than you intended.
- The value is stored in a field or collection that the default models do not track.
- You are using pure data flow when the transformation requires taint tracking.

Temporary bridges

While you investigate, you can add a temporary `isAdditionalFlowStep` that forces the flow across the gap. Once you understand the real relationship, replace the temporary step with a proper model (Chapter 10) so every query benefits.

The path explorer in the VS Code extension is the fastest way to see the last successful step. Use it together with the local-flow probes above.

Exercises

1. Rewrite a query that uses a broad `exists` so that it uses a more direct predicate instead.
2. Take a path query that returns no results and insert an intermediate local-flow check to find where the flow stops.
3. Compare the runtime of two versions of the same query (with and without an extracted predicate) on a moderately large database.

4. (Optional) Use the CodeQL profiler or timing output to identify the slowest predicate in one of your queries.

Solutions and additional examples live in the companion repository under `examples/chapter13/`.

Next Steps

You can now keep queries fast and diagnose the places where flow unexpectedly stops. The next chapter looks at real-world vulnerability patterns and how to turn a single finding into a broader variant analysis.

All examples referenced above live in the companion repository:
github.com/rod-trent/MustLearnCodeQL
under `examples/chapter13/`.

Must Learn CodeQL — Rod Trent
Companion repository: github.com/rod-trent/MustLearnCodeQL